Abstract

Abstract I

This paper presents a strongly-typed, decentralized multiagent simulation — an ant-robot colony — as the computational exemplar of the Research Project Template. Each colony member is an `Agent` that owns exactly one real, on-disk SQLite database and one in-process, fault-injectable protocol endpoint; no agent ever touches another agent's storage or network state. The implementation lives under `projects/templates/template_formal/src/template_formal/`; the demo pipeline is orchestrated by `scripts/02_run_analysis.py`.

Abstract I

The paper's central claim is methodological, not a typing-features showcase: static typing's honest value in Python is edit-time/CI-time error prevention on structurally representable invariants, and nothing more. Nominal identifiers (`AgentId`, `MessageId`, `TxnId` as distinct `NewType` wrappers), a tagged-union `Result[T, E]` ADT with `match`-exhaustiveness, and a session-typed protocol state machine (`IdleSession` → `HandshakingSession` → `EstablishedSession` → `ClosedSession`) each make an illegal program a **type error**, verified by a real `mypy --strict` subprocess run against six known-bad negative-control fixtures plus three known-good positive-control fixtures (`tests/mypy_fixtures/`). Where the

type system cannot help — reusing a consumed transaction handle, reusing a consumed protocol-phase instance, or receiving malformed bytes off an untyped network boundary — the implementation runtime-guards instead, and the manuscript says so explicitly rather than eliding the distinction.

We also frame, without over-claiming, two additional lenses: the per-agent storage schema as a functor $\mathrm{Schema} \to \mathbf{Set}$ in the sense of Fong and Spivak (2018), and each agent's per-tick decision as an approximate minimizer of a closed-form expected-free-energy quantity in the spirit of Friston (2005), bridged to collective organization via the Memory Evolutive Systems framework of (Ehresmann and Vanbremeersch 2007). Both framings are declared as design lenses, not machine-checked mathematical results — the paper is explicit about which of its claims are proofs and which are analogies.

Contributions are architectural, epistemic, and empirical. Architecturally: a zero-mock test suite (`tests/`) covering ADT exhaustiveness, affine-handle reuse, session-type phase transitions, seeded fault injection over a real in-process bus, and a three-agent colony integration test exhibiting a real stigmergic positive-feedback mechanism (deliberately not overclaimed as "emergence" — see sec. 6). Epistemically: an explicit "What mypy –strict proves vs. what is a runtime discipline" section (sec. 3.6) that pins every strong claim to the ISC (Ideal-State Criterion) number of its paired negative-control test, so the claim-to-evidence mapping is auditable rather than asserted. Empirically: eleven pre-registered analyses grouped across three experiment families, falsifiable experiments (sec. 6) — a decay-rate sweep revealing a real, non-monotonic threshold effect (near-zero convergence below decay

≈ 0.35, a 100% plateau at moderate decay, and a measurable decline at total evaporation); a random-choice null-model comparison showing the real mechanism's Wilson-bounded convergence rate (93.3%) does not overlap a chance baseline's (0.67%); and a heterogeneity-magnitude sweep showing convergence rate decreases strictly monotonically as agent preferences spread wider — each stated with its falsification criterion before its real, seeded result, using genuinely new stdlib-only infrastructure (`colony/nullmodel.py`, `colony/sweep.py`) rather than one-off scripts.

Keywords: strongly typed programming, session types, algebraic data types, category theory, Active Inference, multiagent systems, affine types, illegal state unrepresentable.